

(Sack, Small-to-Large Heuristic, DSU on Tree, Heavy-Light Decomposition on Subtrees)

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

```

graph TD
    Root[KỸ THUẬT LẬP TRÌNH MAPREDUCE VÀ CÁC KỸ THUẬT TỐI ƯU HÓA] --> S1[1. Các bài toán MapReduce]
    Root --> S2[2. Tối ưu hóa MapReduce]
    Root --> S3[3. Tối ưu hóa MapReduce]
    Root --> S4[4. Tối ưu hóa MapReduce]
    Root --> S5[5. Các bài toán MapReduce]

    S1 --> S1_1[Đặt vấn đề bài toán:  $(A, m)$  và  $(B, n)$ ]
    S1 --> S1_2[Mô hình bài toán và các thuật toán]
    S1 --> S1_3[Tổng quát hóa thuật toán và các thuật toán]
    S1 --> S1_4[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]

    S2 --> S2_1[Mô hình bài toán và các thuật toán]
    S2 --> S2_2[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S2 --> S2_3[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S2 --> S2_4[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]

    S3 --> S3_1[Mô hình bài toán và các thuật toán]
    S3 --> S3_2[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S3 --> S3_3[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S3 --> S3_4[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]

    S4 --> S4_1[Mô hình bài toán và các thuật toán]
    S4 --> S4_2[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S4 --> S4_3[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S4 --> S4_4[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]

    S5 --> S5_1[Mô hình bài toán và các thuật toán]
    S5 --> S5_2[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S5 --> S5_3[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
    S5 --> S5_4[Ứng dụng của các thuật toán: Set, Map, Vector, Tree, Segment Tree]
  
```

The flowchart is a hierarchical diagram showing the structure of a course or book on MapReduce programming and optimization. It starts with a central title and branches into five main sections, each with its own sub-sections. The sections are: 1. Các bài toán MapReduce, 2. Tối ưu hóa MapReduce, 3. Tối ưu hóa MapReduce, 4. Tối ưu hóa MapReduce, and 5. Các bài toán MapReduce. Each section contains a list of techniques or concepts related to MapReduce programming and optimization.

1

Phương pháp tiếp cận	Độ phức tạp thời gian	Độ phức tạp bộ nhớ	Lý do bế tắc
Gộp <code>std::set</code> ngây thơ (Gộp con vào cha)	$O(N^2 \log N)$	$O(N \log N)$	TLE trên cây suy biến dạng đường thẳng: Cây có dạng $1 - 2 - 3 - \dots - N$. Tại mỗi bước gộp tập kích thước i vào tập kích thước 1, tổng chi phí là $\sum_{i=1}^N i \log i = O(N^2 \log N)$.
Persistent Segment Tree / Fenwick 2D	$O(N \log N)$	$O(N \log N)$	Code cồng kềnh, dễ ăn MLE: Ngốn hàng trăm Megabytes bộ nhớ RAM do phải tạo thêm nút cây phân đoạn động qua từng đỉnh.
Kỹ thuật Gộp Nhỏ Vào Lớn (Small-to-Large)	$O(N \log N)$ hoặc $O(N \log^2 N)$	$O(N)$ (Cực phẳng)	Giải pháp Tối thượng: Đổi chiều gộp dữ liệu hoặc dùng DSU on Tree. Thời gian chạy siêu tốc ($< 0.3s$), bộ nhớ chỉ vài MB.

2. Cơ Sở Toán Học: Bổ Đề Nhân Đôi Kích Thước (Size Doubling Lemma)

2.1. Phát biểu Bổ đề

Bổ đề Nhân đôi Kích thước (Size Doubling Lemma):

Khi gộp hai tập hợp A và B , nếu ta luôn quy ước chuyển toàn bộ phần tử từ **tập có kích thước nhỏ hơn** sang **tập có kích thước lớn hơn**, thì bất kỳ phần tử x nào trong hệ thống sẽ **bị di chuyển tối đa $\lceil \log_2 N \rceil$ lần**.

2.2. Chứng minh toán học

Xét một phần tử x tùy ý thuộc một tập hợp S :

- Ban đầu, phần tử x nằm trong một tập hợp đơn vị gồm 1 phần tử $\implies |S_{\text{ban đầu}}| = 1$.
- Khi phần tử x bị di chuyển (tức là tập chứa x là tập nhỏ hơn trong phép gộp với tập lớn hơn S'):

$$\text{Tập hợp mới } S_{\text{mới}} = S \cup S'$$

Do $|S| \leq |S'|$, ta có:

$$|S_{\text{mới}}| = |S| + |S'| \geq |S| + |S| = 2 \cdot |S|$$

- Mỗi lần phần tử x bị bốc dỡ và chuyển sang tập mới, **kích thước của tập hợp chứa nó tăng lên ít nhất gấp đôi**.
- Vì tổng số phần tử tối đa trong toàn bộ hệ thống là N , kích thước tập hợp không thể vượt quá N :

$$|S| \cdot 2^k \leq N \implies k \leq \log_2 N$$

Trong đó k là số lần phần tử x bị di chuyển.

$$\text{Tổng số lượt di chuyển phần tử qua toàn bộ quá trình} = \sum_x k_x \leq N \log_2 N$$

$$\Rightarrow \text{Độ phức tạp tổng} = \begin{cases} O(N \log N) & \text{nếu thao tác chèn tốn } O(1) \text{ (Vector, Unordered Set)} \\ O(N \log^2 N) & \text{nếu thao tác chèn tốn } O(\log N) \text{ (std::set, std::map)} \end{cases}$$

3. Trường Phái 1: Gộp Con Trỏ Trực Tiếp (Pointer & Set/Map Swapping)

Trường phái này áp dụng khi mỗi đỉnh u thực sự cần duy trì một cấu trúc dữ liệu riêng biệt (`std::set` , `std::map` , `std::vector`) để đại diện cho tập hợp thông tin của cây con nó quản lý.

3.1. Kỹ thuật trao đổi con trỏ trong $O(1)$

Nếu ta khai báo `std::set<int> s[u]` , việc gán `s[u] = s[v]` hoặc copy sẽ tốn $O(|s[v]|)$.

Giải pháp chuẩn C++: Duy trì con trỏ `std::set<int>* s[u]` hoặc dùng trực tiếp hàm `std::swap(s[u], s[v])` (từ C++11, `std::swap` giữa 2 STL containers chỉ trao đổi con trỏ nội bộ trong $O(1)$).

```
void dfs(int u, int p) {
    s[u].insert(color[u]); // Khởi tạo với màu của chính nó

    for (int v : adj[u]) {
        if (v == p) continue;
        dfs(v, u);

        // Kỹ thuật Small-to-Large: Đảm bảo s[u] luôn là tập lớn hơn
        if (s[u].size() < s[v].size()) {
            std::swap(s[u], s[v]); // Trao đổi con trỏ trong O(1)
        }

        // Đổ toàn bộ phần tử từ tập nhỏ s[v] vào tập lớn s[u]
        for (int x : s[v]) {
            s[u].insert(x);
        }
        s[v].clear(); // Giải phóng bộ nhớ của tập con
    }

    ans[u] = s[u].size(); // Trả lời số màu phân biệt trong Subtree(u)
}
```

3.2. Đỉnh cao tối ưu C++17: Hàm `std::set::merge`

Từ chuẩn C++17, thư viện chuẩn hỗ trợ phương thức `s1.merge(s2)` .

Phương thức này **bóc tách trực tiếp các node cây đồ-đen từ `s2` và ghép thẳng vào `s1`** , hoàn toàn **không gọi phép cấp phát bộ nhớ (`new`) hay giải phóng bộ nhớ (`delete`)**:

```
if (s[u].size() < s[v].size()) {
    std::swap(s[u], s[v]);
}
s[u].merge(s[v]); // Ghép node trực tiếp, cực kỳ nhanh!
```

4. Trường Phái 2: DSU on Tree (Thuật Toán Sack)

Mặc dù trường phái 1 rất tiện lợi, nhưng việc sử dụng `std::set` vẫn tốn chi phí hằng số lớn (do cấp phát động con trỏ node đỏ-đen) và độ phức tạp là $O(N \log^2 N)$.

DSU on Tree (Sack) là phát minh tuyệt vời đưa toàn bộ bài toán về $O(N \log N)$ thời gian và $O(N)$ bộ nhớ phẳng, hoàn toàn không dùng bất kỳ `std::set` nào! Toàn bộ hệ thống chỉ sử dụng **DUY NHẤT MỘT mảng tần số toàn cục** `cnt[MAXN]`.

4.1. Khái niệm Con Nặng (Heavy Child) & Con Nhẹ (Light Child)

Cho một đỉnh u có tập các đỉnh con:

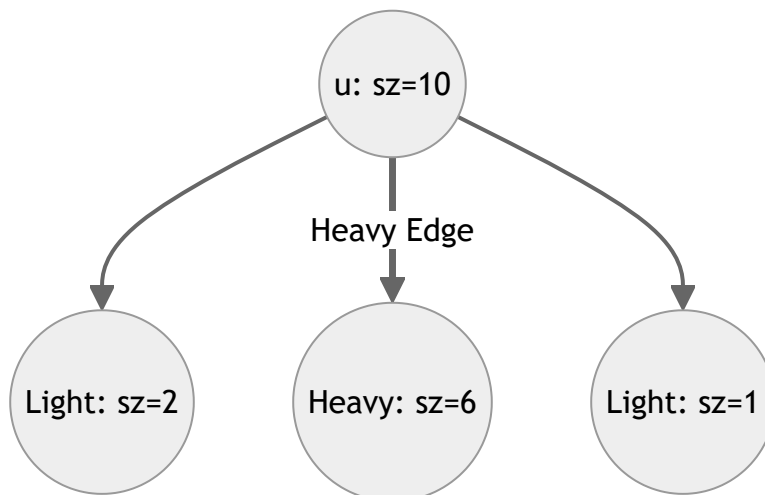
- **Con nặng (Heavy Child):** Là đỉnh con v có kích thước cây con lớn nhất trong tất cả các con của u :

$$\text{heavy}[u] = \arg \max_{v \in \text{children}(u)} \text{sz}[v]$$

- **Con nhẹ (Light Child):** Tất cả các đỉnh con còn lại của u .

i Ghi chú:

****Tính chất then chốt:**** Một đỉnh con nhẹ v luôn thỏa mãn $\text{sz}[v] \leq \frac{\text{sz}[u]}{2}$. Do đó, trên đường đi từ gốc xuống bất kỳ lá nào, ta chỉ có thể gặp tối đa $\lfloor \log_2 N \rfloor$ cạnh nhẹ (Light Edges).



4.2. Quy trình 2 lượt DFS của Thuật toán Sack

DFS Lượt 1: Tiền xử lý kích thước & Tìm con nặng

Duyệt DFS để tính mảng $\text{sz}[u]$ và xác định `heavy[u]`:

```

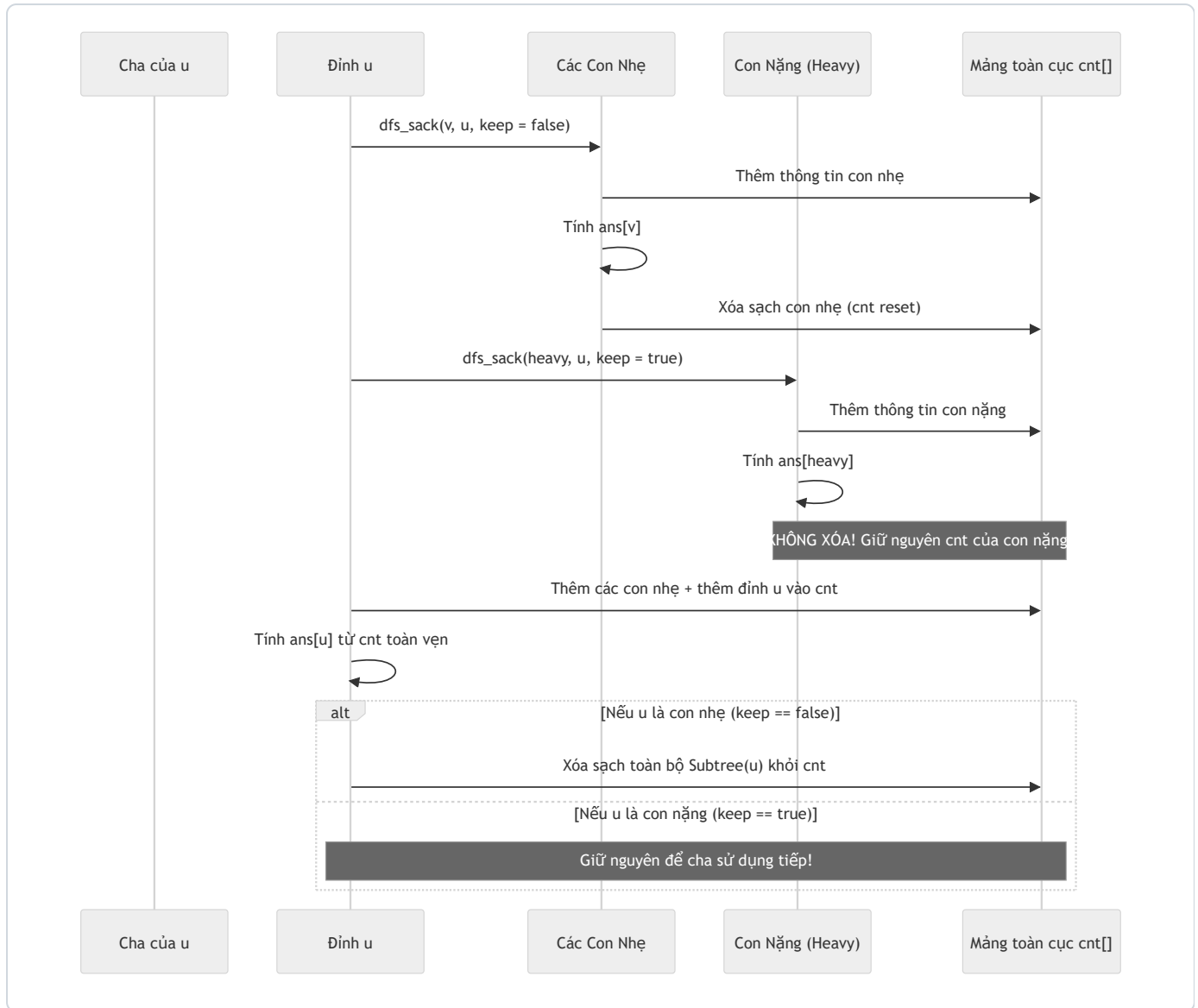
void dfs_size(int u, int p) {
    sz[u] = 1;
    heavy[u] = -1;
    for (int v : adj[u]) {
        if (v == p) continue;
        dfs_size(v, u);
        sz[u] += sz[v];
        if (heavy[u] == -1 || sz[v] > sz[heavy[u]]) {
            heavy[u] = v;
        }
    }
}

```

DFS Lượt 2: Sack Engine với cờ `keep`

Hàm `dfs_sack(u, p, keep)`:

1. **Duyệt các con nhẹ:** Lần lượt gọi `dfs_sack(v, u, keep = false)` cho tất cả các con nhẹ. Sau khi con nhẹ tính xong đáp án của nó, toàn bộ dữ liệu của nó sẽ bị xóa sạch khỏi mảng tần số toàn cục `cnt`.
2. **Duyệt con nặng:** Gọi `dfs_sack(heavy[u], u, keep = true)`. Thông tin của con nặng sẽ được **giữ nguyên trong mảng `cnt`**.
3. **Thêm các con nhẹ vào:** Duyệt lại tất cả các cây con của các con nhẹ và chính đỉnh u , nạp các đỉnh này vào mảng `cnt`. Lúc này, mảng `cnt` chứa đầy đủ thông tin của toàn bộ $Subtree(u)$!
4. **Trả lời truy vấn:** Ghi nhận đáp án cho đỉnh u dựa trên trạng thái của mảng `cnt`.
5. **Dọn dẹp (nếu cần):** Nếu `keep == false`, xóa sạch toàn bộ $Subtree(u)$ khỏi mảng `cnt`.



4.3. Chứng minh Độ phức tạp của DSU on Tree

1. **Con nặng:** Thông tin của con nặng được tính toán và giữ nguyên trong mảng `cnt`. Do đó các đỉnh thuộc cây con nặng không bao giờ bị duyệt lại để nạp vào!
2. **Con nhẹ:** Một đỉnh x chỉ bị duyệt lại và nạp vào mảng `cnt` khi nó là con nhẹ của một tổ tiên nào đó của nó.
3. Vì trên đường đi từ gốc đến x có tối đa $\log_2 N$ cạnh nhẹ, **đỉnh x chỉ bị thêm vào và xóa khỏi mảng `cnt` tối đa $O(\log N)$ lần.**
4. Mỗi thao tác thêm/xóa vào mảng tần số chỉ tốn $O(1)$.

$\Rightarrow$ Tổng thời gian thực thi: $O(N \log N)$

$\Rightarrow$ Bộ nhớ tiêu thụ: $O(N)$ phẳng (hoàn toàn không có chi phí con trỏ!)

5. Cấu Trúc Dữ Liệu Nâng Cao: Segment Tree Merging

Khi thông tin cần duy trì không chỉ là tần số đơn giản mà là một cấu trúc dữ liệu đoạn phức tạp (ví dụ: truy vấn max, sum, phân vị trên cây con), ta kết hợp Small-to-Large với **Hợp nhất Cây Phân Đoạn Động (Segment Tree Merging)**.

5.1. Cơ chế gộp hai cây phân đoạn

Cho hai cây Segment Tree động quản lý đoạn giá trị $[1, V]$ có gốc tại `root_u` và `root_v`:

```
int merge(int u, int v, int l, int r) {
    if (!u || !v) return u ? u : v; // Nếu một trong 2 nút rỗng, trả về nút còn lại

    if (l == r) {
        // Gộp thông tin tại lá
        tree[u].val += tree[v].val;
        return u;
    }

    int mid = l + (r - l) / 2;
    tree[u].lc = merge(tree[u].lc, tree[v].lc, l, mid);
    tree[u].rc = merge(tree[u].rc, tree[v].rc, mid + 1, r);

    pull(u); // Cập nhật lại nút u
    return u;
}
```

5.2. Đánh giá độ phức tạp

- Mỗi thao tác `merge` đi xuống một nút chung giữa 2 cây, và sau khi gộp, một trong hai nút bị xóa bỏ (hòa vào nút kia).
- Ban đầu, mỗi đỉnh u có một cây phân đoạn chứa duy nhất 1 phần tử $\implies$ có $O(N \log V)$ nút trên toàn bộ các cây.
- Mỗi bước đệ quy của hàm `merge` làm giảm số lượng nút của hệ thống đi ít nhất 1.
- Do đó, tổng số lần gọi hàm đệ quy `merge` trên toàn bộ thuật toán không bao giờ vượt quá tổng số nút ban đầu:

Tổng thời gian: $O(N \log V)$

Bộ nhớ: $O(N \log V)$

6. Cạm Bẫy Phòng Thi & Kỹ Thuật Tối Ưu Thực Chiến

6.1. Cạm bẫy truyền tham trị (Pass-by-value) trong STL Set

Rất nhiều thí sinh viết hàm:

```
void dfs(int u, int p, std::set<int> s) // SAI LẦM CHẾT NGƯỜI!
```

Khi truyền tham trị, C++ sẽ copy toàn bộ tập hợp `s` tại mỗi lời gọi đệ quy, biến độ phức tạp thành $O(N^2)$ và lập tức ăn **MLE / TLE**.

Quy tắc Vàng: Luôn truyền tham chiếu `std::set<int>& s` hoặc duy trì mảng con trỏ toàn cục `std::set<int>* s[u]`.

6.2. Cạm bẫy thứ tự xóa trong DSU on Tree

Trong hàm `dfs_sack`:

1. Phải xóa các con nhẹ **ngay sau khi tính xong con nhẹ đó**.
2. **Tuyệt đối không được xóa con nặng!**
3. Khi duyệt xong đỉnh u , nếu `keep == false`, phải xóa sạch cây con của u (bao gồm cả con nặng lẫn các con nhẹ vừa nạp lại).

6.3. Tối ưu Euler Tour cho DSU on Tree

Thay vì viết một hàm `add_subtree(v)` duyệt đệ quy cây con để nạp/xóa, ta có thể kết hợp với mảng làm phẳng Euler Tour:

- Mỗi đỉnh u quản lý đoạn thời gian $[tin[u], tout[u]]$ trên mảng phẳng `euler_node[]`.
- Khi cần nạp toàn bộ cây con của con nhẹ, ta chỉ cần chạy vòng lặp `for (int i = tin[v]; i <= tout[v]; ++i) !`
- Bộ nhớ cache L1/L2 của CPU xử lý mảng tuần tự cực nhanh, tăng tốc độ chương trình lên **gấp 2 đến 3 lần**.

7. Bảng So Sánh Các Kỹ Thuật Xử Lý Truy Vấn Cây Con

Kỹ thuật	Cấu trúc dữ liệu	Khả năng giải bài toán	Độ phức tạp Thời gian	Độ phức tạp Bộ nhớ	Khuyến nghị sử dụng
Small-to-Large Set	<code>std::set</code> / <code>std::map</code>	Tổng quát, truy vấn linh hoạt	$O(N \log^2 N)$	$O(N \log N)$	Khi cần các thao tác set phức tạp (<code>lower_bound</code> , <code>erase</code>)
DSU on Tree (Sack)	Mảng tĩnh <code>cnt[MAXN]</code>	Đếm tần số, max mode, đếm phân biệt	$O(N \log N)$	$O(N)$ (Siêu phẳng)	Lựa chọn số 1 cho hầu hết các bài toán cây con (CSES, Codeforces)
Segment Tree Merge	Cây phân đoạn động	Truy vấn đoạn, phân vị K , đếm cặp	$O(N \log V)$	$O(N \log V)$	Khi bài toán yêu cầu Range Query trên cây con

Kỹ thuật	Cấu trúc dữ liệu	Khả năng giải bài toán	Độ phức tạp Thời gian	Độ phức tạp Bộ nhớ	Khuyến nghị sử dụng
Mo's on Tree (Flattening)	Fenwick / Hai con trỏ	Đoạn đường đi bất kỳ $u \rightarrow v$	$O(N\sqrt{Q})$	$O(N + Q)$	Cho truy vấn trên đường đi , không tối ưu cho cây con

8. Tài liệu Tham khảo & Đọc thêm

1. **Codeforces Community:**
2. [Small-to-Large Merging & DSU on Tree](#) — Bổ đề nhân đôi kích thước $O(N \log N)$.
3. **Competitive Programmer's Handbook** — *Antti Laaksonen*:
4. [Chapter 18: Tree Algorithms](#) — Kỹ thuật gộp cấu trúc dữ liệu nhỏ vào lớn.
5. **VNOI Wiki:**
6. [Kỹ thuật DSU on Tree](#) — Thuật toán Sack và tối ưu bộ nhớ.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.